

Лабораторная работа №23. Интерактивные веб-приложения на JavaScript

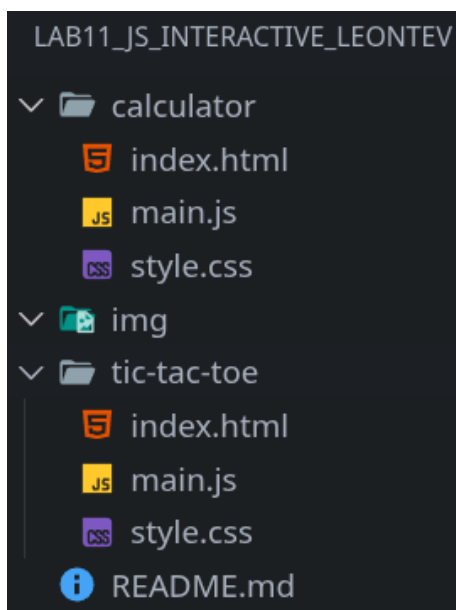
Цель лабораторной работы: Закрепить и систематизировать знания по JavaScript путём разработки интерактивных веб-приложений с использованием HTML, CSS и JavaScript, а также закрепить навыки работы с системой контроля версий Git и удалёнными репозиториями GitHub.

Шаг 1. Подготовка проекта

1. Откройте **Visual Studio Code**
2. Откройте встроенный терминал
3. Перейдите в каталог с документами
4. Создайте директорию для лабораторной работы:

Lab23_JS_Interactive_FIO (где FIO — ваша фамилия)

5. Перейдите в созданную директорию
6. Создайте структуру проекта:



Папки:

- *calculator*
- *tic-tac-toe*
- *img*

Файлы:

- *README.md*

Файлы (внутри папок):

- *index.html*
- *style.css*
- *main.js*

7. Откройте созданную папку в **Visual Studio Code**

Инициализация Git-репозитория

git init

git add .

git commit -m "Initial commit"

Создание удалённого репозитория

1. Создайте **пустой public-репозиторий** на GitHub с именем:

Lab23_JSInteractive

2. Привяжите локальный репозиторий к удалённому:

git branch -M main

git remote add origin <URL_репозитория>

git push -u origin main

4. Сделайте **скриншот терминала** с выполненными командами:

➤ **commit**

➤ **remote add**

➤ **push**

5. В папке **img** сохраните изображение удачного пуша под именем:

gitPushLab23_JS_Interactive_FIO.png

6. Добавьте изменения и отправьте их:

git add .

git commit -m "Added push screenshot and project update"

git push

Проект. Крестики-нолики (Tic-Tac-Toe)

Шаг 2. Создание структуры проекта и интерфейса игры

1. Откройте файл **tic-tac-toe/index.html**

2. Создайте базовую HTML-структуру документа.

```
1  <!DOCTYPE html>
2  <html lang="ru">
3  |   <head>
4  |     <meta charset="UTF-8" />
5  |     <meta name="viewport" content="width=device-width,
6  |     initial-scale=1.0" />
7  |     <title>Крестики-нолики</title>
8  |   </head>
9  |   <body></body>
10 </html>
```

3. Добавьте в **<body>**:

- заголовок страницы (название игры)
- параграф для отображения хода игрока
- основной контейнер игры
- кнопку для начала новой игры

```

8   <body>
9   <h1>Крестики-нолики</h1>
10  <p id="status">Ход игрока: X</p>
11  + <div class="board"></div>
12  <button id="restart">Начать заново</button>
13  </body>

```

4. Добавим игровое поле 3x3

Внутри контейнера создайте **9 одинаковых элементов**, которые будут ячейками игрового поля:

```

11  <div class="board">
12  <div class="cell" data-index="0"></div>
13  <div class="cell" data-index="1"></div>
14  <div class="cell" data-index="2"></div>
15
16  + <div class="cell" data-index="3"></div>
17  <div class="cell" data-index="4"></div>
18  <div class="cell" data-index="5"></div>
19
20  <div class="cell" data-index="6"></div>
21  <div class="cell" data-index="7"></div>
22  <div class="cell" data-index="8"></div>
23  </div>

```

5. Подключим **базовые стили** для калькулятора и JS:

Практическое задание

1. Сделайте скриншот страницы с результатом работы в папку **img** с именем:

step6_tic-tac-toe_HtmlLab23_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 2: added HTML structure for tic-tac-toe"

git push

Шаг 3. Базовые стили

Откройте файл **tic-tac-toe/style.css**

3.1. Стили для всего документа (тег **body**)

```

1  body {
2  |  font-family: Arial, sans-serif;
3  |  text-align: center;
4  }

```

Что делает:

- **Сбрасывает** стандартный шрифт браузера на Arial для единообразия внешнего вида.
- **Центрирует** весь текстовый контент на странице .

Крестики-нолики

Ход игрока: X

Начать заново

Визуальный результат:

После применения этих стилей весь текст на странице будет отображаться **шрифтом Arial** и будет **выровнен по центру**.

3.2. Стили игрового поля (.board)

```
6  .board {
7    display: grid;
8    grid-template-columns: repeat(3, 100px);
9    grid-gap: 5px;
10   justify-content: center;
11   margin: 20px auto;
12 }
```

Что делает:

- **display: grid** создаёт сетчатую структуру для игрового поля 3×3.
- **grid-template-columns: repeat(3, 100px)** — создаёт 3 одинаковые колонки шириной 100 пикселей каждая.
- **grid-gap: 5px** добавляет промежутки между клетками для визуального разделения.
- **justify-content: center** центрирует всё игровое поле по горизонтали.
- **margin: 20px auto** добавляет отступ сверху и снизу 20px, а автоматические боковые отступы также помогают в центрировании.

```
14 .cell {
15   width: 100px;
16   height: 100px;
17   background: #f0f0f0;
18   font-size: 48px;
19   display: flex;
20   justify-content: center;
21   align-items: center;
22   cursor: pointer;
23   user-select: none;
24 }
```

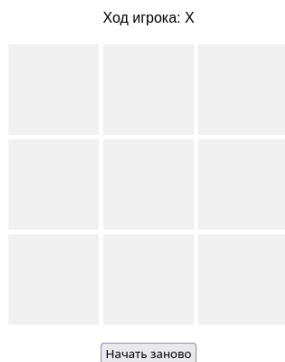
3.3. Стили для каждой клетки (.cell)

width: Ширина клетки
height: Высота клетки (создаёт квадрат)
background: Светло-серый фон клетки
font-size: Размер шрифта для крестиков и ноликов
display: Включаем гибкую вёрстку (Flexbox)
justify-content: Выравниваем содержимое по горизонтали по центру
align-items: Выравниваем содержимое по вертикали по центру
cursor: Меняет курсор на "указатель" при наведении
user-select: Запрещает выделение текста в клетке

Что делает:

- Задаёт **квадратную форму** (100×100 пикселей) для каждой клетки.
- Устанавливает **нейтральный светло-серый фон** для пустых клеток.
- Задаёт **очень крупный размер шрифта** (48px) для чёткого отображения X и O.
- Использует **display: flex с justify-content и align-items** для **идеального центрирования** символов в клетке.
- **cursor: pointer** даёт пользователю **визуальную подсказку**, что клетка кликабельна.
- **user-select: none** **предотвращает выделение текста** при кликах, что улучшает UX.

Крестики-нолики



Визуальный результат:

Каждая клетка будет квадратом 100×100px со светло-серым фоном, при наведении курсор изменится на "руку с пальцем", а символы X/O будут идеально отцентрированы.

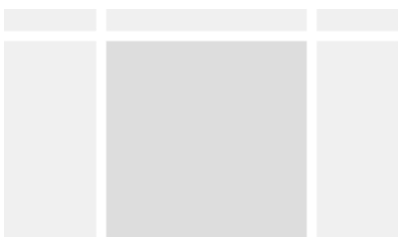
3.4. Эффект при наведении на клетку (.cell:hover)

```
26 .cell:hover {  
27   background: #ddd;  
28 }
```

background: Более тёмный серый фон при наведении

Что делает:

- **Меняет фон клетки** на более тёмный оттенок серого (#ddd) когда пользователь наводит курсор мыши на клетку.
- Создаёт **визуальную обратную связь**, показывая, что клетка активна и готова к клику.



Визуальный результат:

При наведении курсора на любую клетку её фон станет немного темнее, что визуально выделяет активную клетку.

3.5. Стили для кнопки перезапуска (button)

```
30 button {  
31   margin-top: 20px;  
32   padding: 10px 20px;  
33 }
```

margin-top: Отступ сверху 20px (отделяет от игрового поля)

padding: Внутренние отступы: 10px сверху/снизу, 20px слева/справа

Что делает:

- **margin-top: 20px** добавляет **пространство между** игровым полем и кнопкой.
- **padding: 10px 20px** задаёт **внутренние отступы** для кнопки, делая её достаточно крупной для удобного нажатия.

Практическое задание

1. Сделайте скриншот оформленного калькулятора в папку **img** с именем:

step3_tic-tac-toe_CssLab23_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 3: added basic CSS styling for tic-tac-toe"

git push

Шаг 4. Подключение JavaScript

Откройте файл **tic-tac-toe/main.js**

4.1. Получение элементов DOM и инициализация переменных:

```
1 const cells = document.querySelectorAll(".cell");  
2 const statusText = document.getElementById("status");  
3 const restartBtn = document.getElementById("restart");
```

Что делает:

- **cells** — получает все клетки игрового поля (9 элементов) в одну коллекцию.
- **statusText** — получает ссылку на элемент, отображающий статус игры (чей ход, кто победил).
- **restartBtn** — получает ссылку на кнопку перезапуска игры.

```

5 let currentPlayer = "X";
6 let board = ["", "", "", "", "", "", "", "", ""];
7 let gameActive = true;

```

Что делает:

- **currentPlayer** — хранит символ текущего игрока ("X" или "O"). Начинает всегда с "X".
- **board** — массив из 9 пустых строк, представляющий игровое поле. Каждый элемент соответствует клетке.
- **gameActive** — флаг, указывающий, активна ли игра в данный момент (true = игра идёт, false = игра окончена).

```

9 const winConditions = [
10   [0, 1, 2], // Первая горизонталь
11   [3, 4, 5], // Вторая горизонталь
12   [6, 7, 8], // Третья горизонталь
13   [0, 3, 6], // Первая вертикаль
14   [1, 4, 7], // Вторая вертикаль
15   [2, 5, 8], // Третья вертикаль
16   [0, 4, 8], // Диагональ слева-направо
17   [2, 4, 6], // Диагональ справа-налево
18 ];

```

Что делает:

- Создаёт массив всех возможных выигрышных комбинаций.
- Каждый вложенный массив содержит индексы клеток, которые должны быть заполнены одним символом для победы.
- Всего 8 выигрышных линий: 3 горизонтали, 3 вертикали, 2 диагонали.

4.2. Настройка обработчиков событий:

```

20 cells.forEach((cell) => {
21   + cell.addEventListener("click", handleCellClick);
22 });
23
24 restartBtn.addEventListener("click", restartGame);

```

Что делает:

- **Первая часть** — добавляет обработчик клика для каждой из 9 клеток игрового поля.

- **Вторая часть** — добавляет обработчик клика для кнопки перезапуска игры.
- Теперь все интерактивные элементы "ожили" и готовы реагировать на действия пользователя.

4.3. Функция обработки клика по клетке (handleCellClick):

```
26 function handleCellClick(event) {  
27   const cell = event.target; // Получаем кликнутую клетку  
28   const index = cell.getAttribute("data-index"); // Получаем её индекс (0-8)  
29  
30   // Проверка: если клетка уже занята или игра не активна - выходим  
31   if (board[index] !== "" || !gameActive) {  
32     return;  
33   }  
34  
35   // Заполняем клетку символом текущего игрока  
36   board[index] = currentPlayer;  
37   cell.textContent = currentPlayer;  
38  
39   // Проверяем результат хода  
40   checkResult();  
41 }
```

Что делает:

1. **Определяет**, какая именно клетка была нажата.
2. **Получает индекс** этой клетки из атрибута data-index.
3. **Проверяет**, можно ли сделать ход (клетка пуста и игра активна).
4. **Обновляет**:
 - a) Массив board (логическое состояние игры)
 - b) Текст в клетке (визуальное отображение)
5. **Запускает проверку** результата хода.

Логика работы: Пользователь кликает на клетку → если клетка свободна → ставится X или O → проверяется, не победил ли кто-то.

4.4. Функция проверки результата (checkResult):

Обратите внимание, ниже представлена одна большая функция, разъяснение кода разбито на части.


```

43 function checkResult() {
44   let roundWon = false; // Флаг победы
45
46   // Проверяем все возможные выигрышные комбинации
47   for (let i = 0; i < winConditions.length; i++) {
48     const [a, b, c] = winConditions[i]; // Распаковываем индексы клеток
49     +
50     // Проверяем, заполнены ли три клетки одним символом
51     if (board[a] && board[a] === board[b] && board[a] === board[c]) {
52       roundWon = true; // Нашли победную комбинацию!
53       break; // Прерываем цикл, так как победа уже определена
54     }
55   }

```

Что делает (часть 1):

- Перебирает **все 8 выигрышных комбинаций**.
- Для каждой комбинации проверяет: **все три клетки заполнены одним символом** (не пустые и одинаковые).
- Если находит такую комбинацию — устанавливает **roundWon = true** и прекращает проверку.

```

57 // Если кто-то победил
58 if (roundWon) {
59   + statusText.textContent = `Игрок ${currentPlayer} победил!`;
60   gameActive = false; // Останавливаем игру
61   return; // Выходим из функции
62 }

```

Что делает (часть 2):

- Если обнаружена победа:
 - ◆ Обновляет статус с сообщением о победителе.
 - ◆ Останавливает игру (gameActive = false).
 - ◆ Прекращает выполнение функции.

```

64 // Проверяем на ничью (все клетки заполнены)
65 if (!board.includes("")) {
66   statusText.textContent = "Ничья!";
67   gameActive = false; // Останавливаем игру
68   return; // Выходим из функции
69 }

```

Что делает (часть 3):

- Проверяет, **остались ли пустые клетки** в массиве **board**.

- Если пустых клеток нет (все заполнены) — это **ничья**.
- Обновляет статус и останавливает игру.

```
71 // Если игра продолжается - переключаем игрока
72 currentPlayer = currentPlayer === "X" ? "O" : "X";
73 statusText.textContent = `Ход игрока: ${currentPlayer}`;
74 }
```

Что делает (часть 4):

- Если никто не победил и не ничья — **переключает текущего игрока**.
- **Обновляет статус** для отображения, чей сейчас ход.

4.5. Функция перезапуска игры (restartGame):

```
76 function restartGame() {
77   // Сбрасываем состояние игры
78   currentPlayer = "X";
79   board = ["", "", "", "", "", "", "", "", "", ""];
80   gameActive = true;
81   statusText.textContent = "Ход игрока: X";
82   +
83   // Очищаем все клетки на поле
84   cells.forEach((cell) => {
85     cell.textContent = "";
86   });
87 }
```

Что делает:

1. Сбрасывает состояние игры:
 - Начинает с игрока "X"
 - Очищает игровое поле (массив board)
 - Активирует игру
 - Обновляет статус
2. Очищает визуальное поле:
 - Удаляет все символы X и O из клеток

Результат: игра возвращается в исходное состояние, готовое к новой партии.

Практическое задание

1. Сделайте скриншот оформленного калькулятора в папку **img** с именем:
step4_tic-tac-toe_JsLab23_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 4: added basic Js styling for tic-tac-toe"

git push

Самостоятельное задание №1. Создание README.md

Задание: Добавьте описание в README.md.

1. Описание игры:

- Правила игры
- Логика победы/поражения
- Особенности реализации

2. Игровая механика:

- Как определяется победитель?
- Как обрабатываются клики?
- Как работает перезапуск?

3. Структура данных:

- Объяснить массив board
- Объяснить условия победы
- Как хранится состояние игры?

4. Алгоритмы:

- Алгоритм проверки победы
- Алгоритм определения ничьи
- Алгоритм смены игрока

Самостоятельное задание №2. Интерактивная страница «Профиль пользователя»

Создайте самостоятельно приложение Интерактивная страница «Профиль пользователя»

Функционал:

- Форма для ввода имени, возраста, города и хобби
- Валидация (проверка пустых полей)
- Отображение введенных данных на странице

- Кнопка «Очистить данные»
- Минимальное оформление CSS

Пример коммитов:

1. *Initial project: profile page structure*
2. *Added CSS for profile layout*
3. *Implemented form validation*
4. *Added display of user info*
5. *Added clear button functionality*

Скриншот: *profile_FIO.png*

Финальный коммит (после всех шагов):

git add .

git commit -m "Completed Lab 23"

git push